

DATA STRUCTURE AND ALGORITHM (MICROSYLLABUS)

Marks Allocation	Theory	Practical
Sessional	40	50
Final	60	
Total	100	50

Depth Codes

C – Circuit	Dr – Derivation	I – Illustration
D – Definition	Dw – Drawing	N – Numerical
S – State	E – Explanation	P – Proof
Pr – Program	Dm – Demonstration	A – Algorithm

Unit	Topic/Sub topic	Depth Code	Description of Depth	Actual Plan (Hours)	Plan for this semester (Weeks)
1	Introduction		- Define data structures and explain their fundamental role in efficient program design. - Discuss the necessity of data structures by illustrating scenarios where organized data storage is crucial for performance. - Categorize and explain types of data structures (e.g., linear vs. non-linear, primitive vs. non-primitive) and their general characteristics with simple examples	4	1
	1.1. Introduction of Data Structures	D, E	Define Abstract Data Type (ADT) as a mathematical model for data types, emphasizing the separation of interface from implementation.	0.75	
	1.2. Abstract Data Type (ADT)	D, E, I	- Explain the concept of data abstraction and illustrate with simple ADT examples (e.g., Array ADT, List ADT, Stack ADT). - Define and explain various algorithm design techniques, including Brute Force, Divide and Conquer, Greedy Algorithms, Branch and Bound, Backtracking, Randomized, Recursive, and Dynamic Programming.	0.5	
	1.3. Algorithm Design Techniques	D, E, I	Provide conceptual examples for each to illustrate their core idea and typical application domains.	1	

2	Stack and Recursion			D, E, I, Dr	- Define Time and Space Complexity as measures of algorithm efficiency. - Explain Best, Worst, and Average Case Analysis with simple examples, demonstrating how input variations affect performance. - Introduce Rate of Growth and its significance. - Define and explain Asymptotic Notations (Big Oh, Big Omega, and Big Theta) with illustrative examples and simple derivations to analyze the upper, lower, and tight bounds of algorithm performance.	1.75	7	2-3
	2.1. Definition of Stack and its Operations					- Define a stack as a Last-In, First-Out (LIFO) data structure. - Explain its fundamental operations: push, pop, peek (top), isEmpty, and isFull. - Illustrate the behavior of these operations with neat sketches showing the stack state changes.	1	
	2.2. Array Implementation of stack ADT			D, A, Pr	- Implement algorithm for the stack ADT using arrays. - Analyze the time and space complexity of array-based stack operations.	1		
	2.3. Stack Applications			E, A, N, Pr	- Explain and implement algorithms for Expression Conversion (Infix to Postfix and Prefix) and Expression Evaluation (Infix and Postfix) using stacks. - Solve numerical problems involving these conversions and evaluations, demonstrating step-by-step stack usage.	2		
	2.4. Recursion			D, E, I, A, Pr	- Explain the relationship between recursion and stack in terms of function call execution. <i>→ factorial for example, fibonacci</i> - Compare recursion versus iteration in terms of code elegance, memory usage, and performance. - Illustrate the execution of recursive calls using stack frame with trace diagrams. - Discuss types of recursion (e.g., direct, indirect, tail, non-tail recursion), <i>unwinding, non-linear</i> - Implement and solve the Tower of Hanoi problem recursively.	3	5	4-5
3	Queues							
	3.1. Definition of Queue and its Operations			D, E, I	- Define a queue as a First-In, First-Out (FIFO) data structure. - Explain its primary operations: enqueue, dequeue, isEmpty, and isFull. - Illustrate queue behavior with clear diagrams showing element additions and removals.	1		

			3.2. Array Implementation of Queue ADT D, A, Pr	- Implement the queue ADT using arrays. - Analyze the time and space complexity of array-based queue operations, highlighting potential inefficiencies of linear array implementation.	1	
			3.3. Types of Queue ADT D, E, A, I, Pr	- Compare and contrast their operational characteristics and use cases. - Implement and demonstrate the advantages of Circular Queues over Linear Queues for efficient space utilization. - Explain types of queue and perform insertion and deletion from both ends. - Explain priority queue, how elements are managed by priority, and mention real-world applications. - Discuss common applications for each queue type (e.g., CPU scheduling, breadth-first search).	3	
4	Linked List				6	5-6
			4.1. Definition of List and its Operations D, E	Define the general concept of a List ADT and its common operations (e.g., insert, delete, search, traverse).	0.5	
			4.2. Array Implementation of List ADT D, A, I, Pr	- Implement a List ADT using arrays. - Analyze the time complexity of operations like insertion and deletion in the middle of an list.	0.5	
			4.3. Static List and its Limitations D, E, A, I, Pr	Explain the concept of a static list (array-implementation) and explicitly differentiate from array, particularly fixed size and costly insertions/deletions.	1	
			4.4. Linked List (Definition and Operations) D, E, A, I, Pr	- Define a linked list as a dynamic data structure composed of nodes. - Explain its core operations (insertion, deletion, traversal, searching) and illustrate them with neat diagrams showing pointer manipulations. - Implement these operations for a singly linked list. - Analyze their time and space complexity, comparing them to arrays.	0.5	
			4.5. Types of Linked List D, E, A, I, Pr	- Define and explain Singly Linked List, Doubly Linked List, and Circular Linked List. - Compare their structures, advantages, and disadvantages (e.g., bi-directional traversal in doubly linked lists, efficient circular iteration). - Illustrate each type with diagrams and implement key operations for Singly, Doubly and Circular Linked Lists. - Demonstrate and implement how linked lists can be used for Linked List implementation of Stack ADT and Linked List implementation of Queue ADT.	2	
			4.6. Application of Linked List A, I, Pr	Implement the queue ADT using arrays.	1	

5	Tree								
	5.1. Definition and Tree Terminologies	D, E	● Define a tree as a non-linear data structure. ● Explain essential tree terminologies such as root, node, parent, child, sibling, leaf, edge, path, depth, height, level, and subtree with clear diagrams. ● Define Binary Trees and its types (e.g., full, complete, strict, skewed). ● Explain Array and Linked List Representation with examples, highlighting their pros and cons. ● Implement and demonstrate Traversal Algorithms (pre-order, in-order, and post-order traversal). ● Also generate tree from traversal orders. ● Explain the application of Full Binary Tree in Huffman algorithm for data compression, demonstrating its core principles. ● Define Binary Search Tree (BST) and explain its properties (left child < parent < right child). ● Implement fundamental operations on Binary Search Tree including insertion, deletion (various cases), searching, and traversing. ● Demonstrate the construction of Binary Search Tree from a given sequence of elements.	1					
	5.2. Binary Trees	D, E, A, I, Pr	● Explain the application of Full Binary Tree in Huffman algorithm for data compression, demonstrating its core principles. ● Define Binary Search Tree (BST) and explain its properties (left child < parent < right child). ● Implement fundamental operations on Binary Search Tree including insertion, deletion (various cases), searching, and traversing. ● Demonstrate the construction of Binary Search Tree from a given sequence of elements.	2					
	5.3. Binary Search Tree	D, E, I, A, Pr	● Explain the problem with unbalanced binary trees (degeneration to linked list, leading to $O(N)$ worst-case performance). ● Introduce the concept of Balanced Binary Search Tree to maintain logarithmic time complexity. ● Define AVL Tree and explain its necessity (self-balancing). ● Demonstrate construction of AVL tree through insertion and deletion operations, explicitly detailing Rotation Operations (LL, RR, LR, RL) with diagrams and step-by-step rebalancing. ● Provide an introduction to Red-Black Tree, outlining its properties and self-balancing mechanism conceptually. ● Explain the necessity for B-Trees in disk-based systems (minimizing disk I/O).	2					
	5.4. Balanced Binary Tree	E, D, A, I	● Explain the necessity for B-Trees in disk-based systems (minimizing disk I/O). ● Define B-Tree and its properties (multi-way search tree, minimum/maximum degree, all leaves at same level). ● Illustrate the construction of B-Tree through insertion and deletion operations with diagrams, emphasizing node splitting and merging.	1.5					
	5.5. B-Tree	D, E, A, I	● Explain the necessity for B-Trees in disk-based systems (minimizing disk I/O). ● Define B-Tree and its properties (multi-way search tree, minimum/maximum degree, all leaves at same level). ● Illustrate the construction of B-Tree through insertion and deletion operations with diagrams, emphasizing node splitting and merging.	1.5					
	</								

Commented [A1]:

	8	Searching Algorithms			
	7.3. Sorting Algorithms and its efficiency	E, A, I, Pr, N	● Explain the working principle of each specified sorting algorithm: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, Radix Sort, and Heap Sort. ● Implement each algorithm. ● Analyze their time and space complexity (best, worst, average cases) and stability properties. ● Compare their efficiencies and suitability for different data characteristics. ● Solve numerical examples to trace the execution of key algorithms (e.g., Quick Sort, Merge Sort, etc).	4	5
	8.1. Definition of Searching techniques and its Applications	D, E	● Define searching as the process of finding a specific element within a data collection. ● Discuss its widespread applications in various computing contexts (e.g., databases, web search, data retrieval).	0.5	
	8.2. Different Searching algorithms and its efficiency	D, E, A, Pr	● Explain and implement Sequential Search and Binary Search. ● Analyze their time complexity (best, worst, average cases) and preconditions (e.g., sorted data for binary search). ● Solve numerical problems to trace their execution and compare their efficiencies.	1	
	8.3. Hashing	D, E, A, I, Pr, N	● Explain the concepts of Hash Function and Hash Table. ● Define Collision in Hash Table and explain its implications. ● Detail and implement various Collision Resolution Techniques: Chaining Method and Open Addressing (Linear Probing, Quadratic Probing, and Double Hashing). ● Analyze the average-case and worst-case performance of hashing with different collision resolution strategies. ● Solve numerical problems demonstrating hash table operations.	3.5	